

@rag

Appears in: [DL-05](#)

agents/rag.py:57-58, 61-115, 118-134

"ChromaDB for the vector store — local file persistence, pip install, built-in deduplication by content hash." ...
footnote ¹ — "the code is @rag."

@rag

agents/rag.py:57-58

```
57 def _content_hash(text: str) -> str:
58     return hashlib.sha256(text.encode()).hexdigest()[:16]
```

...

agents/rag.py:61-115

```
61 def ingest_files(file_paths: list[str]) -> dict:
62     """Ingest documents into ChromaDB. Returns counts of ingested/skipped chunks."""
63     try:
64         collection = _get_collection()
65         total_ingested = 0
66         total_skipped = 0
67
68         for file_path in file_paths:
69             try:
70                 text = _extract_text(file_path)
71                 if not text.strip():
72                     continue
73
74                 chunks = _chunk_text(text)
75                 if not chunks:
76                     continue
77
78                 doc_hash = _content_hash(text)
79                 source = os.path.basename(file_path)
80                 candidate_ids = [f"{doc_hash}_{i}" for i in range(len(chunks))]
81
82                 # Deduplication: skip already-stored IDs
83                 existing = collection.get(ids=candidate_ids)
84                 existing_ids = set(existing["ids"])
85
86                 novel_ids = []
87                 novel_chunks = []
88                 novel metas = []
89                 for i, (chunk_id, chunk) in enumerate(zip(candidate_ids, chunks)):
90                     if chunk_id not in existing_ids:
91                         novel_ids.append(chunk_id)
92                         novel_chunks.append(chunk)
93                         novel metas.append({"source": source, "chunk_index": i})
94
95                 total_skipped += len(chunks) - len(novel_chunks)
96
97                 if novel_chunks:
98                     embeddings = _embed(novel_chunks)
99                     collection.add(
100                         ids=novel_ids,
101                         embeddings=embeddings,
102                         documents=novel_chunks,
103                         metadatas=novel metas,
104                     )
105                     total_ingested += len(novel_chunks)
106
107             except Exception as e:
108                 # Per-file errors are logged but don't abort the whole batch
109                 print(f"[rag] Error ingesting {file_path}: {e}")
110
111         return {"ingested": total_ingested, "skipped": total_skipped}
112
113     except Exception as e:
114         print(f"[rag] ingest_files error: {e}")
115         return {"ingested": 0, "skipped": 0, "error": str(e)}
```

...

agents/rag.py:118-134

```
118 def retrieve(query: str) -> list[str]:
119     """Retrieve top-k relevant chunks for a query string."""
120     try:
121         collection = _get_collection()
122         if collection.count() == 0:
123             return []
124
125         embeddings = _embed([query])
126         results = collection.query(
127             query_embeddings=embeddings,
128             n_results=min(config.RAG_TOP_K, collection.count()),
129         )
130         return results["documents"][0] if results["documents"] else []
131
132     except Exception as e:
133         print(f"[rag] retrieve error: {e}")
134         return []
```

What it shows: The retrieval layer: content-hash chunk IDs give free deduplication (re-upload → skipped: N, ingested: 0); retrieve() returns the top-k chunks injected into the text agents.

Source: agents/rag.py:57-58, 61-115, 118-134 · image rendered by build_snippets.py